

WebRelayX

WebRelayX - SecCoreGmbH, GitHub.

- Published: date not stated
- Original: <<https://github.com/SecCoreGmbH/WebRelayX>>
- Preserved from: <https://github.com/SecCoreGmbH/WebRelayX> (github-repository-api) on 2026-10-02
- Repository commit: ef223a48523e5a785d2ce2cd848e962fa0737df4
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from documentation in [SecCoreGmbH/WebRelayX](#), pinned to commit [ef223a48523e](#). GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

README.md

[View original document](#)

WebRelayX

WebRelayX is an NTLM relay tool focused on Web (http/s) targets. It builds on `impacket's ntlmrelayx` and adds cookie harvesting and auth scan.

The goal of this tool is to automate NTLM-Relaying to targets other than ADCS or Exchange, by harvesting session cookies before or after successful authentication.

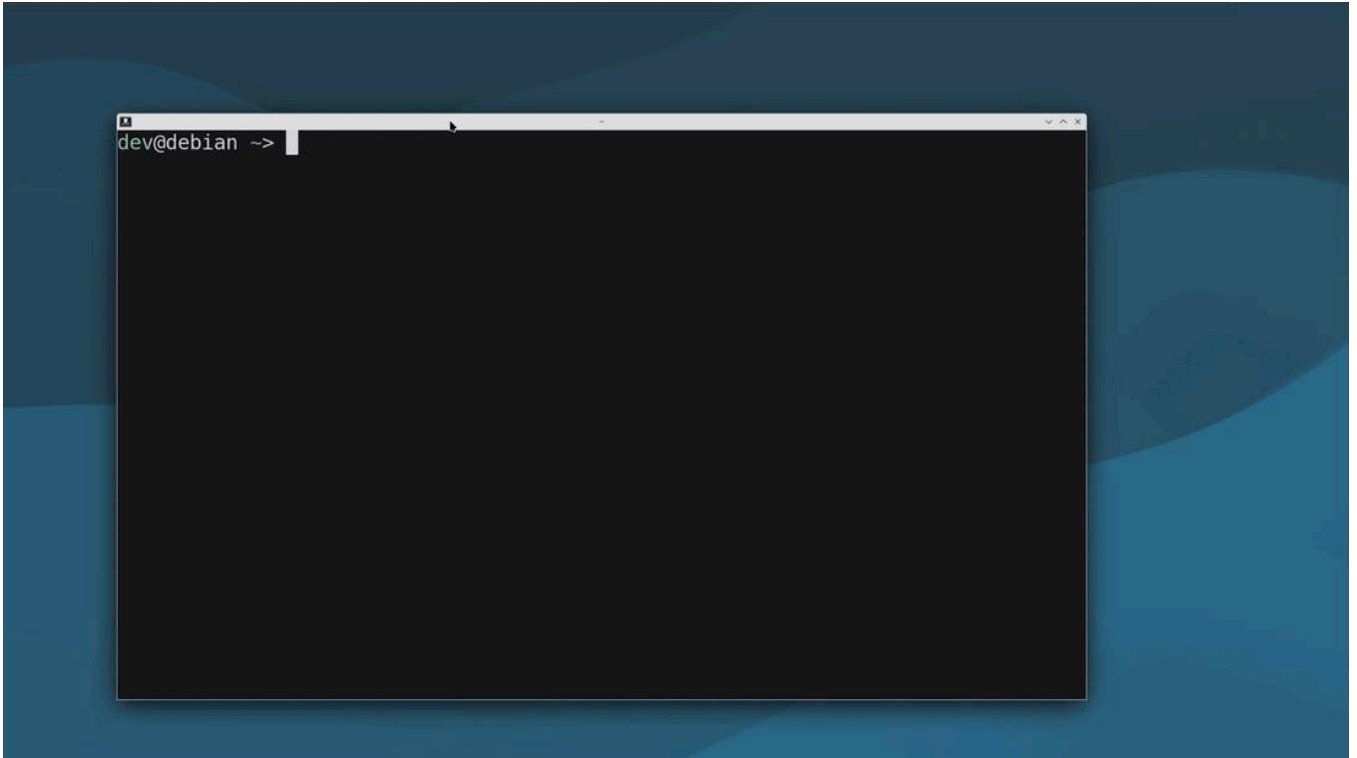
It checks web pages for NTLM auth and automatically stores cookies that are set before or after an authentication takes place. After a successful relay, the tools checks the captured cookies for validity and stores them in `cookies.jsonl` if they are valid. a browser with those cookies can also be spawned later with the `launch` subcommand.

This tool is for educational and authorized testing purposes only. Do not use it without permission. We are not responsible for any misuse.

Created and maintained by [SecCore GmbH](#).

Demo

Scanning and Relaying to a target webserver:



Requirements

- Python 3.13+
- [impacket](#) >= 0.13
- root user to bind to privileged ports (445, 80, etc.)

Installation

```
pipx install git+https://github.com/SecCoreGmbH/WebRelayX
```

Or clone and install locally:

```
git clone https://github.com/SecCoreGmbH/WebRelayX
cd WebRelayX
pipx install .
```

Or with Poetry:

```
poetry install
```

Playwright

We use playwright to automate browser interactions for `--open-browser` and `launch` subcommands. This tool attempts to install playwright and the browser binaries automatically if they are not found. This may take some time and will only be done once, after that it should detect playwright automatically.

Subcommands

scan

Probes one or more HTTP/HTTPS targets for NTLM authentication and reports:

- If auth is required and NTLM can be used for authentication
- NTLM domain, hostname, and DNS info from the challenge
- MIC and SPN enforcement flags
- EPA status **(WIP!)**
 - I still need some more test data and understanding about how to detect EPA reliably. May not work correctly!

```
webrelayx scan -t http://intranet.demo.internal
webrelayx scan -t http://intranet.demo.internal -t https://sharepoint.demo.internal
webrelayx scan -tf targets.txt
```

relay

Starts relay listeners (SMB, HTTP, WCF, RAW, RPC, WinRM) and relays incoming NTLM authentication to the specified HTTP/HTTPS targets. Captured session cookies are printed to stdout and optionally written to a file.

```
# basic relay, outputs cookies to terminal
sudo webrelayx relay -t http://intranet.demo.internal

# relay without SMB or WCF server
sudo webrelayx relay -t http://intranet.demo.internal --no-smb-server --no-wcf-server

# automatically open a browser with the injected session cookies
sudo webrelayx relay -t http://intranet.demo.internal --open-browser
```

The browser stays open until you close it, even after webrelayx exits.

list

Lists captured sessions from cookies.jsonl with index, user and target. Use this index to launch with `launch` subcommand.

```
webrelayx list
```

launch

launches a browser with the cookies from a saved session in cookies.json. Use `list` to get the cookie index.

```
# launch first session with default browser
webrelayx launch -i 0
# launch second session with firefox
webrelayx launch -i 1 --browser firefox
```

Options

Common

Flag	Description
<code>-t</code> , <code>--target</code>	Target URL (repeatable)
<code>-tf</code> , <code>--targets-file</code>	File with one target URL per line
<code>-v</code> , <code>--verbose</code>	Enable debug logging

relay

Flag	Default	Description
<code>-l</code> , <code>--listen-ip</code>	<code>0.0.0.0</code>	IP to bind all listeners to
<code>--no-smb-server</code>	-	Disable SMB listener
<code>--no-http-server</code>	-	Disable HTTP listener
<code>--no-wcf-server</code>	-	Disable WCF listener
<code>--no-raw-server</code>	-	Disable RAW listener
<code>--no-rpc-server</code>	-	Disable RPC listener
<code>--no-winrm-server</code>	-	Disable WinRM/WinRMS listeners
<code>-b</code> , <code>--open-browser</code>	-	Open browser with captured cookies after relay
<code>--browser</code>	<code>chromium</code>	Select browser to launch cookie injection with

launch

Flag	Default	Description
<code>-i</code> , <code>--index</code>	-	Session from cookie jsonl

Flag	Default	Description
<code>--browser</code>	<code>chromium</code>	Select browser to launch cookie injection with

This product includes software developed by SecureAuth Corporation (<https://www.secureauth.com/>) and Fortra (<https://www.fortra.com>).

Archived from <https://github.com/SecCoreGmbH/WebRelayX>. Rendered offline from the local Markdown copy.